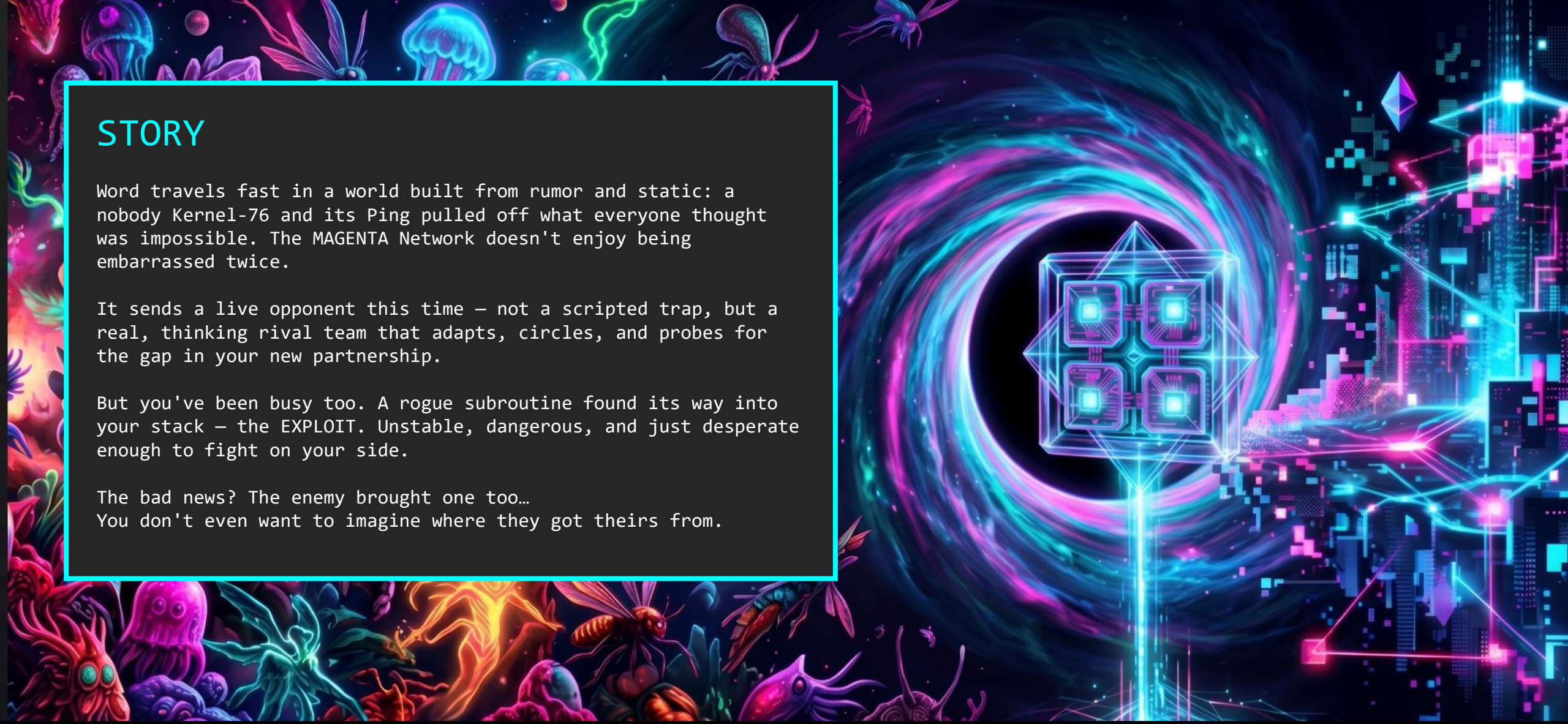


STORY

Word travels fast in a world built from rumor and static: a nobody Kernel-76 and its Ping pulled off what everyone thought was impossible. The MAGENTA Network doesn't enjoy being embarrassed twice.

It sends a live opponent this time — not a scripted trap, but a real, thinking rival team that adapts, circles, and probes for the gap in your new partnership.

But you've been busy too. A rogue subroutine found its way into your stack — the EXPLOIT. Unstable, dangerous, and just desperate enough to fight on your side.

The bad news? The enemy brought one too…
You don't even want to imagine where they got theirs from.

CLOUDFLIGHT CODING CONTEST

#WeLoveSoftware
u too?
cloudflight.io/career

COLONY: ONE

LEVEL 3 – KERNEL POWER

Your objective:

Train a model, upload it and win one match. You play CYAN against a greedy bot: it makes the most valuable capture it can, otherwise a random move.

Results vary between runs; losing only costs time.

The next slides show what you downloaded and what your model must look like.



ADDITIONAL RULES

NEW PIECE “EXPLOIT”

Move: L-shaped jump (2 + 1 tiles). It leaps over all pieces but cannot land on a friendly piece.
Capture: by landing on an enemy piece.

Guards: with several guards next to a Kernel, the Ping is hit before the Exploit.

In this level you play with just Kernel, Ping and Exploit. The downloaded environment contains more pieces, but they are not used in Level 3.



WHAT YOU DOWNLOADED

colony-one-env.zip holds the exact game engine and search our server uses:
README.md – start here: install, run a game, check a model.
docs/rules.md docs/observation.md docs/submission.md – the rules, the board encoding (16 planes) and the model contract.
colony_one/ – the game engine and the search (MCTS); scene.json is the starting position.
tools/validate_submission.py – runs the same checks as the server. Run it before every upload.
play/model_vs_model.py – let two models play each other.
play/random.onnx – a valid example model.

SETUP

Python 3.11, 3.12, 3.13 or 3.14 (64-bit). In the unzipped folder: python -m venv .venv, activate it, then pip install -e .
Python 3.10 and Intel Macs cannot install the required onnxruntime.
No training framework is included: use PyTorch, TensorFlow, JAX… anything that exports ONNX.

WHAT IS MCTS?

Monte-Carlo Tree Search: the search that picks the moves. It simulates possible continuations and asks your model two things about each position – which moves look promising, and how good the position is.



YOUR MODEL

One file, model.onnx: ONNX opset 13–18, weights inside the file, at most 5 MiB.
Input (one): float32 (batch, 16, 8, 8) – the board seen from the side to move, as 16 planes (docs/observation.md).
Output 1, policy: float32 (batch, 4096) – a raw score per action, action = from_tile × 64 + to_tile. Illegal actions are ignored.
Output 2, value: float32 (batch) between -1 and 1 – how good the position is for the side to move.
The batch dimension must be dynamic, with the same name on the input and both outputs.

LIMITS

Standard layers only: convolutions, linear layers, activations, normalization, pooling and reshapes with fixed shapes. No loops, no if, no data-dependent shapes.
Checking a model must take under 3 seconds, a Level 3 match under 20 seconds. Small networks are far below that.
One upload every 30 seconds per team.

WHEN A MODEL IS REJECTED

The page shows the reason. tools/validate_submission.py gives the same message locally.

If you get the "Model dry run exceeds the 3-second limit" error, please try again.

The full specification, with the complete list of allowed layers and the common export fixes, is in docs/submission.md.

CLOUDFLIGHT CODING CONTEST

#WeLoveSoftware
u too?
cloudflight.io/career

COLONY: ONE





CLOUDFLIGHT
CODING
CONTEST

#WeLoveSoftware
u too?
cloudflight.io/career